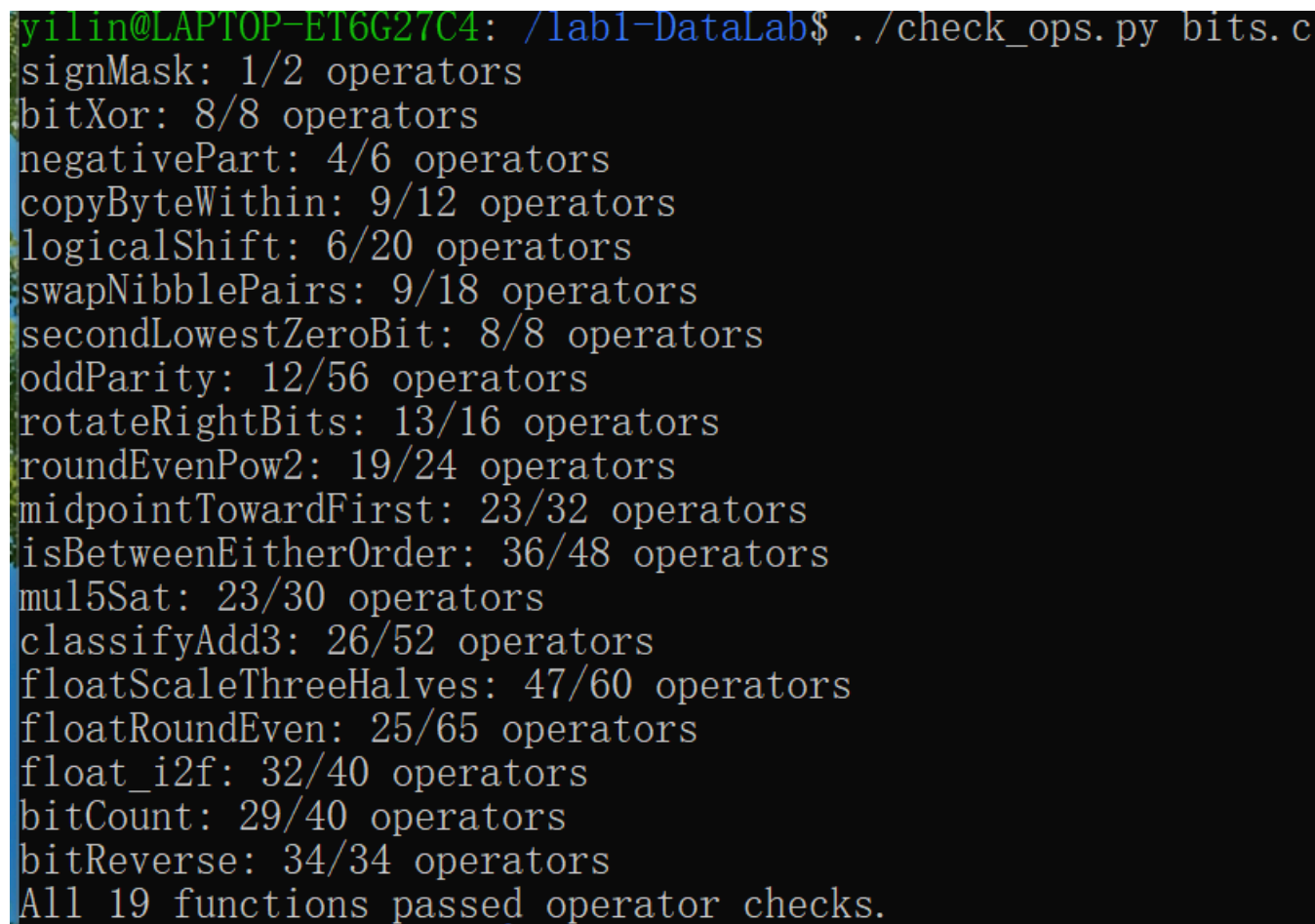


Lab 1: Data Lab实验报告

姓名：郭怡琳 学号：25300680194

1.实验结果展示



```
yilin@LAPTOP-ET6G27C4: /lab1-DataLab$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 13/16 operators
roundEvenPow2: 19/24 operators
midpointTowardFirst: 23/32 operators
isBetweenEitherOrder: 36/48 operators
mul5Sat: 23/30 operators
classifyAdd3: 26/52 operators
floatScaleThreeHalves: 47/60 operators
floatRoundEven: 25/65 operators
float_i2f: 32/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

在终端中执行 ./check_ops.py bits.c 进行合法性检查的截图

```

yilin@LAPTOP-ET6G27C4: /lab1-DataLab$ ./btest
Score    Rating  Errors  Function
1         1        0      signMask
2         2        0      bitXor
3         3        0      negativePart
4         4        0      copyByteWithin
4         4        0      logicalShift
4         4        0      swapNibblePairs
4         4        0      secondLowestZeroBit
5         5        0      oddParity
5         5        0      rotateRightBits
5         5        0      roundEvenPow2
5         5        0      midpointTowardFirst
7         7        0      isBetweenEitherOrder
7         7        0      mul5Sat
7         7        0      classifyAdd3
7         7        0      floatScaleThreeHalves
10        10        0      floatRoundEven
10        10        0      float_i2f
10        10        0      bitCount
10        10        0      bitReverse
Total points: 110/110

```

在终端中执行 ./btest 进行正确性测试的截图

2. 函数实验思路

P1 signMask。 从 1 左移 31 位，得到唯一最高位为 1、其余位为 0 的掩码。依据本实验的位向量机器模型。检查结果应为 0x80000000。

P2 bitXor。 两个不同位才输出 1，可以写成 $(x \& \sim y) \mid (\sim x \& y)$ ；由于 $\mid$ 被禁止，按德摩根律改为 $\sim(\sim(x \& \sim y) \& \sim(\sim x \& y))$ 。

P3 negativePart。 $x \gg 31$ 在负数时为全 1、非负数时为 0。先令 $a = x \& (x \gg 31)$ ，使非负数变成 0、负数保留原值，再用 $\sim a + 1$ 取补码相反数。因此，非负数返回 0，一般负数返回其绝对值。对于 INT_MIN，其数学绝对值超出 32 位有符号整数范围；依据本实验的 32 位补码位向量模型，结果仍为 0x80000000，即 INT_MIN。

P4 copyByteWithin。 字节号 i 对应起始位 $8i$ 。先把源字节右移到低端并与 0xff 取出，再构造目的字节掩码，把旧目的字节清零，最后把源字节左移到目的位置合并。

P5 logicalShift。 算术右移 $x \gg n$ 后，高位可能补 1。表达式 $\sim(((1 \ll 31) \gg n) \ll 1)$ 生成低 $32-n$ 位为 1 的掩码；与右移结果相与即可清掉补入的符号位。

- P6 swapNibblePairs。** 用重复模式 `0x0f0f0f0f` 选出每个字节的低四位并左移 4 位；原高四位右移 4 位后与同一掩码相与；最后合并即可。
- P7 secondLowestZeroBit。** 用 `z=~x` 把零位变为 1；用 `z&(~z+1)` 找到最低的 1，异或删除它后再取一次最低的 1。
- P8 oddParity。** 依次把高半段与低半段异或：移位 16、8、4、2、1。最后最低位等于 1 的个数 mod 2。题目要求偶数个 1 时返回 1，所以对最低位取逻辑非。
- P9 rotateRightBits。** 先取 `k=n&31` 避免移位量越界；右移部分沿用 P5 的逻辑右移掩码，另一部分把 `x` 左移 `(32-k)&31` 位后合并。
- P10 roundEvenPow2。** $x = q \cdot 2^n + r$ ，其中 $q = x \gg n$ ， $0 \leq r < 2^n$ ， r 可由低 n 位取得。当 $n \geq 1$ 时，将 r 与半单位 $2^{(n-1)}$ 比较：大于时令 q 加 1，小于时保留 q ；恰好相等时，仅在 q 为奇数时加 1，使结果为偶数。。
- P11 midpointTowardFirst。** 用 `(x&y)+((x^y)>>1)` 计算 `floor((x+y)/2)`。当精确中点是半整数且 $x > y$ 时，结果应向上取整，额外加 1；否则保留向下取整。
- P12 isBetweenEitherOrder。** 分别求 $x \geq a$ 、 $x \geq b$ ，符号不同时根据符号直接判断，同号时通过差的符号判断。当 x 位于两个端点之间时，异或得到 1；在区间外时，异或得到 0。端点相等的情况不能全部由异或覆盖，最后再合并 $x == a$ 、 $x == b$ 的判断，使两个端点都包含在区间内。
- P13 mul5Sat。** 先算 `four=x<<2`、`five=four+x`。若 $x \gg 29$ 不等于 $x \gg 31$ ，则乘 4 已溢出；否则再以加数与结果的符号判断第二步是否溢出。任一溢出时按原始 x 的符号选择 `INT_MAX` 或 `INT_MIN`，用全 0/全 1 掩码完成选择。
- P14 classifyAdd3。** 设 32 位低位和为 $r = x + y$ 、 $s = r + z$ 。每一步若两加数同号而结果变号，就说明发生了溢出：正溢出记 +1，负溢出记 -1，两次修正累加得 `high`。两次修正相加可能抵消，所以需要把两次修正都累加完再判断。总修正为正时返回 1，负时返回 -1，为零时返回 0。
- P15 floatScaleThreeHalves。** 先处理 NaN/无穷与符号。非规格化数把尾数乘 3 后除 2，按中点向偶数舍入，结果可能跨入 `exp=1`；规格化数先恢复隐含首位，乘 3，根据最高位选择右移 1 位或 2 位，相应调整阶码，再对丢弃位做像偶数舍入。尾数舍入到 2^{24} 时再右移 1 位并把阶码加 1；阶码达到 255 时返回对应符号的无穷。
- P16 floatRoundEven。** `exp<126` 时绝对值小于 0.5，返回带原符号的零；`exp=126` 时，等于 0.5 的端点选零，其余选带符号的 1；`exp>=150` 时数值已经是整数或是特殊值，原样返回。中间的 127..149 由 `150-exp` 算出小数位数，清除小数位后比较丢弃部分与半单位；刚好相等时检查保留部分最后一位的奇偶。
- P17 float_i2f。** 零单独处理。提取符号后，在无符号类型中求绝对值覆盖 `INT_MIN`。定位最高的 1，指数等于该位置加 127。若最高位位置不超过 23，则左移对齐；否则右移得到 24 位有效数，对被丢弃的位作向偶数舍入。若舍入进位形成 2^{24} ，有效数右移 1 位，阶码加 1。
- P18 bitCount。** 用掩码 `0x55555555` 先把相邻 1 位的计数合并成 2 位组；用 `0x33333333` 合成 4 位组；用 `0x0f0f0f0f` 合成字节计数。再依次右移 8、16 位相加，低 6 位就是 0-32 的答案。
- P19 bitReverse。** 依次交换 16 位半字、8 位字节、4 位半字节、2 位组、单个位。掩码 `0x00ff00ff` 可以由两个 `0xff` 构造；再由 $m^8(m8<<4)$ 、 $m^4(m4<<2)$ 、 $m^2(m2<<1)$ 推出后三个掩码。

3.AI使用说明

将 AI 用于思路梳理与实验报告语言润色；不用于直接生成可提交代码。最终代码由本人独立实现与调试，并自行验证。